UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 02 — RPC, timeouts, retries e idempotencia

Departamento de Computación
Ingeniería Informática – Comisión A – Pilar
Equipo docente – 11 de agosto de 2026

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Comunicación en sistemas distribuidos

Objetivos de la clase

- Entender una comunicación distribuida como una interacción no inmediata.
- Razonar qué puede estar ocurriendo mientras el cliente todavía no observa respuesta.
- Identificar las capas que intervienen en una llamada remota.
- Analizar cómo cada capa puede detectar, ocultar, transformar o amplificar una falla.
- Conectar timeout, retry e idempotencia con decisiones de mitigación.

Problema real

En sistemas distribuidos:

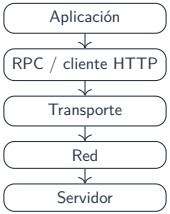
- todo ocurre mediante mensajes
- la red es incierta
- no hay garantía de entrega

Pregunta

¿Qué significa “la operación ocurrió”?

Comunicar no es instantáneo

Una operación remota atraviesa varias capas, y cada una puede responder distinto ante una falla.



- una capa puede reintentar mientras otra ya venció su timeout
- una capa puede ocultar pérdida y otra observar demora
- el servidor puede ejecutar aunque el cliente ya haya abandonado

RPC (Remote Procedure Call)

Abstracción:

- llamada local: función
- llamada remota: red

RPC intenta ocultar la existencia de la red. En un sistema robusto se gestiona la red, no se oculta.

Problema clave

¿La operación se ejecutó o no?

- request perdido
- response perdido
- servidor lento
- servidor caído

Timeout

El cliente decide:

- esperar
- reintentar
- informar error

Clave

Timeout no implica falla: solo indica que no hubo respuesta dentro de una ventana.

Retry

Retry introduce:

- recuperación ante fallas transitorias
- duplicación potencial
- presión adicional sobre el sistema

Ejemplo:

- pago duplicado si el primer cobro ocurrió y se perdió la respuesta

Idempotencia

Definición:

Aplicar la operación múltiples veces produce el mismo resultado observable.

Ejemplo:

- PUT idempotente
- POST no necesariamente
- cobro con request id único como idempotencia implementada

Semánticas de entrega

- at-most-once: evita duplicación, puede perder ejecución
- at-least-once: evita pérdida, puede duplicar
- exactly-once: garantía lógica costosa, no magia de la red

Trade-off

- confiabilidad versus complejidad
- latencia versus corrección
- retry versus duplicación
- idempotencia versus más estado

Cierre

Idea clave

Comunicar en sistemas distribuidos es decidir bajo incertidumbre y reconstruir qué pudo haber ocurrido en cada capa.